AI-Powered Jewellery Feedback Calling System

Complete Technical Code Flow & Architecture Guide: Purchase-to-Call Execution

System: Jewellery Customer Feedback AI Agent	**Architecture:** MERN + Durable Dual Worker
Telephony Providers: Ringg.ai / Retell AI / Mock	**Calling Policy:** TRAI Hours (10 AM - 8 PM IST)
Non-Negotiable Constraint: Max 2 Attempts / Zero Duplicate Calls	**Languages:** Malayalam, English, Manglish

1. Executive Overview & Lifecycle Summary

When a jewellery purchase is entered (via the manual bill form in the admin UI or the secured POS billing API), the application guarantees an automated, reliable post-purchase feedback call to the customer while strictly enforcing legal telecom regulations and customer privacy rules. The full lifecycle traverses six distinct stages:

Phase	Component / Action	Description & Safeguard
1. Ingestion	AddPurchaseModal.jsx purchaseController.js	Customer details received; phone normalized to E.164 (+91); idempotent deduplication.
2. Scheduling	purchaseService.js FeedbackProcess Model	Checks Opt-Out Registry; calculates call delay (0 mins for immediate, 15 mins by default); c
3. Lock & Claim	callingService.js (claimProcessForCalling)	Acquires 5-minute atomic database lease lock to prevent race conditions across parallel wo
4. Eligibility	callingEligibilityService.js	Pre-flight verification: 1) Opt-out flag, 2) Explicit consent, 3) Valid phone, 4) Attempt limit <=
5. Dial Dispatch	CustomAIProvider.js Ringg.ai / Retell AI	Transmits structured payload (customer name, items, bill ID) via authenticated REST API w
6. Ingestion Webhook	webhookService.js Feedback & Complaint Models	Receives call outcome, validates rating (1-5 strictly), flags complaints for rating <= 2, and qu

2. Detailed Code Path & File-by-File Walkthrough

Below is the exact execution flow through every function, file, and database collection.

Step 1: Frontend Bill Entry & API Dispatch

File: Frontend/src/components/purchases/AddPurchaseModal.jsx
• The staff member enters Customer Name, Mobile Number, Invoice Number, Amount, and Items Summary.
• **Initial Call Delay:** Defaults to Immediate (0 min) for testing, or 15 Minutes as standard store policy.
• Upon clicking 'Create Purchase & Schedule Call', an authenticated HTTP POST request is sent to /api/v1/purchases with bearer JWT.

Step 2 & 3: Purchase Ingestion, Normalization & Idempotency

File: Backend/src/services/purchaseService.js → createPurchaseAndSchedule()
1. **Phone Normalization:** Calls normalizePhoneNumber() (using Google's *libphonenumber-js*) to format raw inputs like 8086939423 into canonical E.164 (+918086939423).
2. **Duplicate Protection:** Searches Purchase.findOne({ $or: [{ invoiceNumber }, { externalPurchaseId }] }). If found, returns existing record immediately to prevent duplicate bill ingestion.
3. **Opt-Out Check:** Queries OptOutRegistry.exists({ normalizedPhoneNumber }). If customer previously asked not to be called, sets isOptedOut = true.
4. **Record Creation:** Creates Purchase record and linked FeedbackProcess with scheduledAt = purchaseDate + delayMinutes.
5. **Automated Call Trigger:**
   • If delayMinutes === 0: Dispatches setImmediate(() => executeCallAttempt(feedbackProcess._id, 'auto-on-purchase')) to dial immediately in the background.
   • If delayMinutes > 0: Calls scheduleFeedbackCallJob() to enqueue BullMQ delayed job.

Step 4: Dual-Engine Scheduler (BullMQ + Durable MongoDB Reconciliation)

Files: Backend/src/server.js & Backend/src/workers/callingWorker.js
• Outbound calling runs on a dual-engine architecture: BullMQ with Redis for production queueing, coupled with a durable MongoDB reconciliation loop running every 10 seconds inside the server.
• Every 10 seconds, reconcileDueCalls() queries:
  { callStatus: { $in: ['scheduled', 'retry_scheduled'] }, scheduledAt: { $lte: now } }
• This ensures zero lost calls even if Redis restarts, server reboots, or calls are scheduled for later hours.

Step 5: Pre-Flight Safeguards & Worker Lease Locking

File: Backend/src/services/callingService.js → executeCallAttempt()
1. **Atomic Lock:** claimProcessForCalling() issues an atomic findOneAndUpdate setting callStatus = 'queued' and a 5-minute lease lock. If another worker attempts to claim it simultaneously, it receives null and skips cleanly.
2. **Eligibility Verification (callingEligibilityService.js):**
   a. Customer in OptOutRegistry? → Status becomes opted_out; call aborted.
   b. No explicit consent? → Status becomes skipped; call aborted.
   c. Attempt count >= 2? → Status becomes failed (max attempts reached).
   d. **TRAI Legal Calling Window (10:00 AM - 08:00 PM IST):** If outside permitted hours, shifts call to 10:15 AM next morning.
3. **Call Attempt Initialized:** Creates a CallAttempt record in database and updates status to calling.

Step 6 & 7: Telephony Provider Dispatch & AI Voice Call

File: Backend/src/providers/voice/CustomAIProvider.js (Ringg.ai Adapter)

1. Assembles payload with required parameters:
   name: 'Feedback Call - Customer Name', mobile_number: E.164 phone, agent_id: Ringg agent UUID, from_number_id: caller ID, variables: customer and invoice metadata.
2. Dispatches HTTP POST to Ringg.ai API (https://prod-api.ringg.ai/ca/api/v0/calling/outbound/individual) with header X-API-KEY: [rg_live_key].
3. Telephony server dials customer's phone over PSTN telecom network.
4. **Conversational Agent Behavior:** Speaks in natural Kerala Malayalam or polite Indian English. Confirms 2 minutes availability, asks for overall rating (1 to 5), asks optional follow-up question, handles complaints neutrally without making false promises, and logs Do-Not-Call requests.

Step 8: Inbound Webhook Processing & Rating Validation

File: Backend/src/services/webhookService.js → processWebhookEvent()

When the phone call finishes, the AI telephony provider delivers a webhook payload to POST /api/v1/webhooks/voice-provider/events.

1. **Idempotency Check:** Verifies event_id / call_id against WebhookEvent collection to discard duplicate deliveries.
2. **AI Rating Sanitation (Rule 7):** AI model output is treated as untrusted. Non-integer or out-of-range values (<1 or >5) are forced to null.
3. **Automated Complaint Escalation:** If customer gives a rating ≤ 2 or reports an issue, the system automatically creates a high-priority ticket in Complaint collection for store staff review.
4. **Opt-Out Handling:** If customer asked not to be called again, their phone is permanently inserted into OptOutRegistry.
5. **Retry Policy:** If outcome was no_answer or busy and attempt count < 2, schedules **Attempt 2 of 2** for 30 minutes later.

3. State Transition Matrix

State	Trigger / Transition Condition	Next Possible States
pending	Purchase ingested via POS API; process created.	scheduled, skipped
scheduled	Initial state waiting for call time (15m or 0m).	queued, skipped, opted_out
queued	Atomic lease acquired by worker instance.	calling, scheduled (lease expired)
calling	Dispatched to Voice Telephony Provider API.	completed, no_answer, busy, failed
completed	Feedback conversation finished; rating collected.	TERMINAL STATE
no_answer / busy	Call unanswered; if attempt < 2, schedules retry.	retry_scheduled, failed (max attempts)
retry_scheduled	Eligible temporary failure; waiting 30 minutes.	queued (on attempt 2)
opted_out	Customer requested Do-Not-Call suppression.	TERMINAL STATE (Suppressed forever)
skipped	Ineligible at pre-flight (no consent, out of hours).	TERMINAL STATE (or deferred to next day)

4. Core Backend Architecture & Responsibilities Map

File Path	Primary Responsibility	Key Functions / Exports
src/services/purchaseService.js	Bill ingestion, normalization & initial call scheduling	createPurchaseAndSchedule() getPurchasesList()
src/services/callingService.js	Worker lease locking, pre-flight checks & call dispatch	executeCallAttempt() claimProcessForCalling() rescheduleFeedbackProcess()
src/services/callingEligibilityService.js	TRAI 10am-8pm window, consent & opt-out evaluation	evaluateCallingEligibility() isWithinCallingHours()
src/providers/voice/CustomAIProvider.js	Telephony adapter for Ringg.ai / Vapi / Bland API	initiateCall() normalizeWebhookEvent() verifyWebhookSignature()
src/services/webhookService.js	Webhook ingestion, rating sanitation & complaint ticket	processWebhookEvent() simulateCallResult()
src/workers/callingWorker.js	Durable 10s MongoDB reconciliation & BullMQ worker	reconcileDueCalls() startReconciliationTimer()